

Name: Sanskriti

USN: 2VX24UE048

Exp_7(a)

Title: 0/1 Knapsack Problem using Dynamic Programming

```
In [2]: # Function to solve the 0/1 Knapsack Problem
def DP_KnapSack(capacity, weight, profit, n):
    # Initialize a 2D table K[n+1][capacity+1] with zeros
    K = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Build the table K[][] in a bottom-up manner
    for i in range(n + 1):
        for w in range(capacity + 1):
            if i == 0 or w == 0:
                K[i][w] = 0
            elif weight[i-1] <= w:
                # Max of including the item or excluding it
                K[i][w] = max(profit[i-1] + K[i-1][w - weight[i-1]], K[i-1][w])
            else:
                # Item cannot fit, carry over previous value
                K[i][w] = K[i-1][w]

    return K[n][capacity]

# --- Taking inputs from user ---
n = int(input("Enter the number of items: "))

print("Enter the weight and profit for each item:")
weight = []
profit = []

for i in range(n):
    w = int(input("Weight of item {}: ".format(i + 1)))
    p = int(input("Profit of item {}: ".format(i + 1)))
    weight.append(w)
    profit.append(p)

capacity = int(input("Enter the capacity of the Knapsack: "))

# Calculate and print the maximum profit
max_profit = DP_KnapSack(capacity, weight, profit, n)
print("=====")
print("Maximum Profit: ", max_profit)
print("=====")
```

```
Enter the weight and profit for each item:
=====
Maximum Profit:  220
=====
```

In []: